

Morelord Character Export

Module documentation reference

READMEs and user / GM documentation

September 7, 2026 | Local source snapshot

Local manifest version: **0.3.3**. This PDF collects the source documents listed below. Original manual version labels are preserved; older manuals may not cover newer README changes. This conversion does not establish current public release availability.

Included source	Document
morelord-character-export/README.md	Morelord Character Export

Prepared in the shared Morelord document style. Use PDF bookmarks to navigate sections.

Morelord Character Export

Morelord Character Export adds a **Morelord Export** command to the title-bar controls of D&D 5e character sheets in Foundry Virtual Tabletop.

The command downloads a portable JSON file for import into **My Characters** on the [Morelord Gaming website](#) after registering for a free account. The export contains the Actor's complete source data, a snapshot of values already prepared by Foundry and D&D 5e, and an embedded image library for character-sheet artwork.

Compatibility

See the GM and player guide for installation, exporting, importing into My Characters, and troubleshooting. The website publishes this guide at `/docs/morelord-character-export` after its documentation deployment.

- **Foundry VTT:** v14 only.
- **Game system:** D&D 5e.
- Uses the Foundry v14 `getHeaderControlsApplicationV2` hook.
- Designed for the current D&D 5e ApplicationV2 character sheet and Tidy 5e character sheet.
- Foundry v13 and legacy ApplicationV1 sheets are not supported.

Current features

- Foundry VTT v14 support.
- D&D 5e character Actors only.
- ApplicationV2 title-bar integration.
- Complete serializable `Actor#toObject(true)` source data, including embedded Items, item activities, and Active Effects.
- Display-ready values copied from the live, prepared Actor under `derived`.
- Deduplicated embedded image library containing:
 - character portrait
 - prototype token image
 - class, subclass, species, background, feat, spell, equipment, container, tool, consumable, and other Item icons
- Item IDs mapped directly to their embedded image asset IDs.
- Images resized and converted to WebP when possible.
- Public API for macros and development testing.
- Exact Foundry and D&D 5e system versions recorded in every export.

Installation during development

Place the repository outside the Foundry data directory and create a Windows directory junction inside your Foundry modules folder.

Run Command Prompt as Administrator:

```
mklink /J "C:\Path\To\FoundryVTT\Data\modules\morelord-character-export"
"C:\Path\To\GitHub\morelord-character-export"
```

Enable **Morelord Character Export** under **Manage Modules** in the desired world.

Use

Open a D&D 5e player-character sheet and choose **Morelord Export** from the sheet's title-bar controls.

The downloaded filename uses this pattern:

```
character-name.morelord-character.json
```

Macro API

Export an Actor directly:

```
await game.modules.get("morelord-character-export").api.exportCharacter(actor);
```

Export the currently selected token's Actor:

```
const actor = canvas.tokens.controlled[0]?.actor;
if (!actor) return ui.notifications.warn("Select a token first.");
await game.modules.get("morelord-character-export").api.exportCharacter(actor);
```

Export by UUID:

```
await game.modules
  .get("morelord-character-export")
  .api.exportActorByUuid("Actor.ACTOR_ID");
```

Export format

Format version 3 stores all images once and references them by asset ID:

```
{
  "format": "morelord-character",
  "formatVersion": 3,
  "exportedAt": "2026-08-03T00:00:00.000Z",
  "source": {
    "moduleVersion": "0.3.0",
    "foundryVersion": "14.365",
    "systemVersion": "5.3.3"
  },
  "actor": {},
  "derived": {},
  "assets": {
    "images": {
      "image-1": {
        "path": "tokenizer/pc-images/example.webp",
        "mimeType": "image/webp",
        "data": "data:image/webp;base64,...",
        "embedded": true
      }
    },
    "references": {
      "actor": {
        "portrait": "image-1",
```

```

    "prototypeToken": "image-2"
  },
  "items": {
    "ITEM_ID": "image-3"
  }
},
"summary": {
  "requested": 3,
  "embedded": 3,
  "failed": 0
}
}
}

```

The raw Actor and embedded Item records retain their original Foundry image paths. Importers should resolve portable images through `assets.references` and `assets.images`, then use the original path or a generic icon only as a fallback.

Installation from Foundry

In Foundry VTT Setup, open **Add-on Modules**, choose **Install Module**, and paste this manifest URL:

<https://raw.githubusercontent.com/tmoreland72/morelord-character-export/main/module.json>

After installation, enable **Morelord Character Export** under **Manage Modules** in a D&D 5e world.

Previous versions are available from the repository's GitHub Releases page.

Standard release workflow

Character Export uses Core's shared `release.ps1`, with module-specific packaging and publishing values in `release.config.json`. The next prepared release is **0.3.3**; the release script advances `module.json` from 0.3.2 when publishing.

Before publishing:

- Run `node --test test/character-exporter.test.mjs` and syntax-check both files in `scripts/`.
- Verify export from the supported Foundry v14 D&D 5e and Tidy 5e sheets, then import the file into My Characters using a free website account.
- Review `RELEASE-NOTES-0.3.3.md` and commit the preparation changes on `main`; the release script requires a clean working tree.
- Keep `docs/README.md` frontmatter and instructions aligned with the release version. The release script validates this guide and includes `docs/` in the ZIP. The website's product-docs registry and deployment checkouts must include Character Export; a successful website release publication then requests a documentation deployment.
- Ensure the package is registered with Foundry and GitHub CLI is authenticated (`gh auth login`).
- Set `RELEASE_PUBLISH_TOKEN` and `FOUNDRY_RELEASE_TOKEN` in the process environment or the ignored project `.env` file. Never commit credentials.

Validate the standard workflow:

```
.\release.ps1 -Version 0.3.3 -DryRun
```

Publish when ready:

```
.\release.ps1 -Version 0.3.3
```

The workflow validates the repository and release notes, packages only the configured runtime files, validates the ZIP, updates the manifest, commits and tags the release, pushes to GitHub, and publishes to GitHub Releases, Foundry VTT, and the Morelord Gaming release feed. The archive is `morelord-character-export.zip` in the repository root with `module.json` at the ZIP root. The stable installation manifest remains on `main`; Foundry release submissions use the version-tagged manifest.

-DryRun validates a temporary archive without publishing or advancing the source manifest; it still requires a clean repository, remote access, and publishing token configuration. For local packaging checks without publishing credentials, use `-SkipWebsitePublish -SkipFoundryPublish` with `-DryRun`. Draft and prerelease modes skip Foundry and the public website feed. Use `-WebsiteOnly` only to retry the website step after the GitHub release exists.